

它本质上是在用可视化方式解释 Transformer：一个把“注意力机制”作为核心、能够高效并行处理文本序列的模型架构。

一句话总结

Transformer 的核心思想是：模型在处理每个词时，不是只按顺序一个词一个词读，而是让每个词主动“观察”句子里的其他词，判断哪些词对理解自己最重要，然后并行完成编码和生成。

通俗地说，Transformer 像一个阅读能力很强的人：看到一句话时，不是机械地从左到右记住每个字，而是会自动判断“这个代词指的是谁”“这个形容词修饰什么”“哪些词之间有关联”。

1. Transformer 的整体结构：Encoder + Decoder

讲义一开始用机器翻译举例：输入一句法语，例如：

Je suis étudiant

模型输出英文：

I am a student

Transformer 整体可以看成两个部分：

1. Encoder 编码器

负责理解输入句子，把原句变成一组包含语义信息的向量。

2. Decoder 解码器

负责根据 Encoder 的理解结果，一个词一个词生成目标语言句子。

原始论文中，Encoder 和 Decoder 都是堆叠结构，论文中各堆了 6 层，但 6 不是神秘数字，只是一种架构选择。

通俗解释：

Encoder 像“阅读理解模块”，Decoder 像“写作输出模块”。先读懂原文，再生成译文。

2. Encoder 内部结构：Self-Attention + Feed Forward

每个 Encoder 主要有两个子层：

2.1 Self-Attention：让每个词看见其他词

Self-Attention 是 Transformer 的核心。

例如句子：

The animal didn't cross the street because it was too tired.

这里的 “it” 指的是 animal 还是 street？

人类很容易判断是 animal，因为 “tired” 更像描述动物，而不是街道。Self-Attention 的作用就是帮助模型在处理 “it” 时，把注意力放到 “animal” 和 “tired” 等相关词上。

通俗解释：

Self-Attention 就像模型在心里问：

“我现在要理解这个词，那句子里哪些词对它最重要？”

所以，Transformer 不是孤立地理解每个词，而是让每个词根据上下文重新理解自己。

2.2 Feed Forward：对每个位置做进一步加工

Self-Attention 之后，每个词对应的向量会进入一个前馈神经网络。

这个网络对每个位置单独处理，但所有位置使用同一套网络参数。

通俗解释：

Self-Attention 负责“看上下文”，Feed Forward 负责“加工理解结果”。

3. 词如何进入模型：Embedding

计算机不能直接理解文字，所以每个词要先变成向量，这一步叫 **Embedding**。

例如：

Thinking Machines

会被转换成两个向量，每个词一个向量。

在原始 Transformer 中，词向量维度是 512。也就是说，每个词被表示成一个包含 512 个数字的向量。

通俗解释：

Embedding 就是给每个词做一张“语义身份证”。

比如 “cat” 和 “dog” 的向量会更接近，因为它们语义相近；“cat” 和 “stock market” 的向量会更远。

4. Self-Attention 的计算逻辑：Q、K、V

讲义最重要的部分是解释 Self-Attention 怎么算。

每个输入词向量都会被转换成三个向量：

1. **Query**，查询向量 **Q**
2. **Key**，键向量 **K**
3. **Value**，值向量 **V**

它们的直觉可以这样理解：

- **Query**：我现在这个词想找什么信息？
- **Key**：我这个词能提供什么线索？
- **Value**：如果别人关注我，我实际提供什么内容？

Self-Attention 的计算过程大致是：

1. 当前词的 **Query** 和所有词的 **Key** 做点积，得到相关性分数。
2. 分数除以一个缩放值，原文中是除以 **key** 维度平方根。
3. 用 **Softmax** 把分数变成权重，所有权重加起来等于 1。
4. 用这些权重加权求和所有词的 **Value**。
5. 得到当前词的新表示。

通俗解释：

假设模型正在理解 “it”。

它会拿着 “it” 的 **Query** 去问全句所有词：

“你和我有多相关？”

如果 “animal” 的相关性高，它的权重就大；如果 “street” 的相关性低，它的权重就小。最后，“it” 的新向量就会吸收更多 “animal” 的信息。

5. Attention 公式的核心含义

讲义中给出了 Self-Attention 的矩阵形式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

这句话可以拆成三层：

1. **QK^T**：计算每个词和其他词的相关性。
2. **除以 $\sqrt{d_k}$** ：防止数值过大，让训练更稳定。
3. **softmax 后乘 V**：根据相关性权重，汇总真正的信息内容。

通俗解释：

这就像开会时，每个人都在听别人发言，但不是平均听，而是根据“谁对我当前任务最有帮助”来分配注意力。

6. Multi-Head Attention：多个注意力头

Transformer 不只做一次 Self-Attention，而是同时做多组注意力计算，这叫 **Multi-Head Attention**。

原始 Transformer 使用 8 个 attention heads。

为什么要多个头？

因为一句话中可能存在多种关系：

- 一个头关注代词指代关系；
- 一个头关注主谓关系；
- 一个头关注修饰关系；
- 一个头关注位置关系。

例如处理 “it” 时，一个注意力头可能关注 “animal”，另一个注意力头可能关注 “tired”。

通俗解释：

一个注意力头像一个阅读视角。

多头注意力就是让模型从多个角度同时读一句话。

这很像我们读文章时，不只看“谁做了什么”，还会看“原因是什么”“代词指谁”“情绪是什么”“逻辑转折在哪里”。

7. Positional Encoding：让模型知道词的顺序

Transformer 的一个特点是可以并行处理所有词，但这也带来一个问题：

如果模型同时看所有词，它怎么知道词的顺序？

例如：

dog bites man
man bites dog

词一样，但顺序不同，意思完全不同。

所以 Transformer 要给每个词向量加上 **Positional Encoding**，位置编码。

位置编码会告诉模型：

- 这是第 1 个词；
- 这是第 2 个词；
- 这个词和那个词相隔多远。

原始 Transformer 使用正弦和余弦函数生成位置编码。

通俗解释：

Embedding 告诉模型“这个词是什么意思”，Positional Encoding 告诉模型“这个词出现在什么位置”。

两者加起来，模型才知道：

这个词是什么 + 它在句子里的位置。

8. Residual Connection 和 Layer Normalization：让深层模型更稳定

讲义还提到 Encoder 和 Decoder 每个子层周围都有：

1. **Residual Connection**，残差连接
2. **Layer Normalization**，层归一化

残差连接的意思是：

模型不是只用于子层处理后的结果，而是把原始输入也加回来。

通俗解释：

残差连接像是给模型留一条“高速公路”。如果某一层学得不好，原始信息还能直接传下去，不至于在深层网络中丢失。

Layer Normalization 则是让每一层的数值更稳定，减少训练过程中的震荡。

通俗解释：

Residual Connection 负责“别把原信息弄丢”，LayerNorm 负责“让数值别乱飞”。

9. Decoder 的特殊之处：不能偷看未来

Decoder 负责生成输出句子。它和 Encoder 类似，也有 Self-Attention 和 Feed Forward，但 Decoder 多了一个关键限制：

Decoder 在生成当前位置的词时，不能看到未来的词。

例如生成：

I am a student

当模型正在生成 “am” 时，它只能看到已经生成的 “I”，不能提前看到后面的 “a student”。

这通过 **masking** 实现，也就是把未来位置遮住。

通俗解释：

这就像考试时不能偷看答案。

模型必须根据已经生成的内容和输入句子的编码结果，预测下一个词。

10. Encoder-Decoder Attention：Decoder 如何看原文

Decoder 中还有一个特殊层：**Encoder-Decoder Attention**。

它的作用是让 Decoder 在生成每个目标词时，去关注输入句子的相关部分。

例如翻译 “Je suis étudiant”：

当 Decoder 要生成 “student” 时，它应该重点看输入里的 “étudiant”。

通俗解释：

Encoder 先把原文读懂；Decoder 写译文时，每写一个词都会回头看原文中最相关的部分。

这就是翻译时的“对齐”能力。

11. 输出层：Linear + Softmax，把向量变成词

Decoder 最后输出的是一个向量，但我们需要的是具体词。

所以 Transformer 最后会经过：

1. **Linear 层**：把向量映射到词表大小的 logits。
2. **Softmax 层**：把 logits 转成概率分布。

3. 选择概率最高的词作为输出。

例如词表有 10,000 个词，模型会给每个词一个分数，然后 Softmax 转成概率。概率最高的词就是当前时间步输出的词。

通俗解释：

模型最后不是直接“写出一个词”，而是在整个词表里做选择题：

下一个词最可能是谁？

12. 训练过程：预测错了就调整参数

训练时，模型会拿输入和标准答案对比。

例如输入：

merci

目标输出：

thanks

一开始模型参数是随机的，可能给 “thanks” 的概率很低。训练过程会计算模型输出和正确答案之间的差距，这个差距就是 **loss**。

然后通过反向传播调整参数，让模型下次更接近正确答案。

通俗解释：

训练就像做题：

1. 模型先猜答案；
2. 和标准答案对比；
3. 算出错得有多离谱；
4. 修改内部参数；
5. 重复很多次，逐渐变准。

对于完整句子，模型不是只预测一个词，而是一步一步预测：

I → am → a → student → <eos>

其中 <eos> 表示句子结束。

13. 解码方式：Greedy Decoding 和 Beam Search

讲义最后提到生成时可以用不同策略。

Greedy Decoding

每一步都选概率最高的词。

优点：简单、快。

缺点：可能局部最优，不一定整句最好。

Beam Search

每一步保留多个候选路径，而不是只保留一个。

例如 beam size = 2，就是始终保留两个最可能的翻译候选，继续往后比较。

通俗解释：

Greedy Decoding 像每一步都选眼前最优；

Beam Search 像同时保留几条可能路线，走几步后再看哪条整体更好。

14. 这篇讲义最值得掌握的核心框架

你可以把 Transformer 理解成下面这条流程链：

Input tokens → Embedding → Positional Encoding → Encoder Self-Attention → Encoder Feed Forward → Decoder Masked Self-Attention → Encoder-Decoder Attention → Decoder Feed Forward → Linear → Softmax → Output tokens

更通俗地说：

文字先变成向量 → 加上位置信息 → 每个词观察上下文 → 形成输入理解 → 生成端逐词输出 → 每一步参考原文 → 最后从词表中选词。

15. 对大模型理解的启发

这篇讲义虽然讲的是原始 Transformer，但它对理解今天的大语言模型非常关键。

现在的 GPT、Claude、DeepSeek 等模型，本质上都建立在 Transformer 机制之上，只是具体结构、训练规模、数据、推理优化方式发生了大量演进。

尤其要抓住三点：

第一，Transformer 的核心不是“翻译”，而是“序列建模”

虽然讲义用机器翻译举例，但 Transformer 可以用于任何序列任务：

- 文本生成；
- 摘要；
- 问答；
- 代码生成；
- 多模态理解；
- Agent 推理。

第二，Attention 是上下文理解能力的来源之一

模型之所以能处理代词、长距离依赖、语义关联，是因为每个 token 可以根据上下文重新计算自己的表示。

第三，并行化是 Transformer 成功的重要原因

相比 RNN 一个词一个词顺序处理，Transformer 可以并行处理整句话，因此更适合大规模训练。

这也是为什么 Transformer 后来成为大模型时代的基础架构。

最后用一个比喻总结

可以把 Transformer 想象成一个多人会议系统：

- 每个词都是一个参会者；
- Query 是“我想知道什么”；
- Key 是“我能被别人如何识别”；
- Value 是“我真正贡献的信息”；
- Attention 是“谁应该听谁多一点”；
- Multi-Head Attention 是“从多个角度同时开会”；
- Positional Encoding 是“每个人坐在哪个位置”；
- Decoder 是“根据会议纪要逐字写出最终答案”。

所以 Transformer 的强大之处在于：

它不是机械地读文本，而是在每个 token 之间建立动态关系网络。